



PENETRATION TESTING REPORT

Mediroza General Hospital

Web Application Security Assessment

Prepared by: Muhammad Ahsan

Cybersecurity Mentor: Waqas Karim, CCIE

Organisation: Networkwalks

Batch: Week 4 Capstone Project

Target: <https://medirozahospital.com>

Classification: **Confidential**

This report is produced as part of a controlled educational exercise by Networkwalks. The target has been authorised for security testing. These techniques must never be applied to any system without explicit written permission from the owner.

1. Executive Summary

I was assigned to conduct a black-box penetration test on the web infrastructure of Mediroza General Hospital at <https://medirozahospital.com>. Written authorisation for this assessment was provided by the client via Networkwalks. The goal was to identify vulnerabilities, demonstrate their real-world impact through controlled exploitation, and provide recommendations to improve the organisation's security posture.

During the assessment I identified seven vulnerabilities ranging from Medium to Critical severity. The most significant finding was a SQL Injection vulnerability on the patient portal login page, which allowed me to bypass authentication entirely without knowing any valid credentials. This gave me access to three confidential patient lab report PDFs. After cracking the PDF encryption using password-cracking tools, I discovered sensitive metadata inside one of the files that pointed to a forgotten database backup stored in a publicly accessible folder on the server. This backup contained the monthly salaries of all 30 hospital employees and the shareholding details of 10 shareholders.

The overall security posture of the target is poor. Multiple critical vulnerabilities exist that would allow an unauthenticated attacker to access confidential patient data, staff financial records, and corporate ownership information with minimal effort and no specialised equipment.

Overall Risk Rating: CRITICAL — Immediate remediation is recommended.

2. Scope and Methodology

2.1 Scope

The assessment was limited to the following target as agreed with the client.

Target domain: <https://medirozahospital.com>

The following were excluded from scope: social engineering, denial-of-service attacks, and any testing outside the agreed domain.

2.2 Methodology

I followed a structured black-box penetration testing methodology consisting of four phases.

- **Reconnaissance:** Passive and active information gathering using publicly available information and command-line/network tools.
- **Vulnerability Identification:** Analysing the application's behaviour to find weaknesses in authentication and input handling.
- **Exploitation:** Demonstrating the real impact of each vulnerability through controlled exploitation.
- **Documentation:** Recording all findings, evidence, and remediation recommendations in this report.

2.3 Testing Environment

- Oracle VirtualBox — virtualization platform hosting the isolated testing environment
- Kali Linux 2026.2 — dedicated offensive security testing operating system
- Mozilla Firefox — manual browser-based testing, DOM inspection, and login-form interaction

2.4 Tools Used

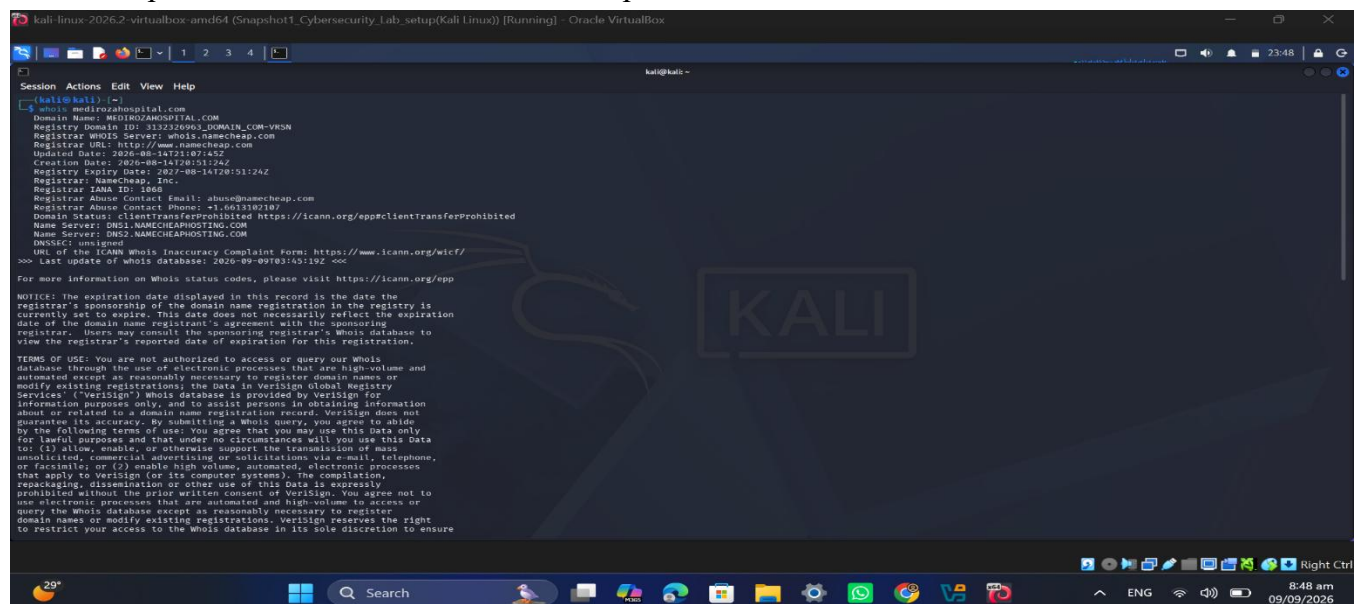
- WHOIS / nslookup / DNSRecon — domain and DNS infrastructure reconnaissance
- curl — command line tool for sending HTTP requests and reading web server responses
- Nmap / Zenmap — network port scanning and service/version detection
- Gobuster — directory and file enumeration on the web root
- robots.txt Analysis — identifying hidden directories and sensitive routes
- Networkwalks Hash Calculator — for extracting crackable password hashes from PDF files
- Networkwalks Password Cracker — for cracking PDF password hashes using wordlists
- ExifTool — for reading hidden metadata from PDF files
- wget — for downloading files from the web server

3. Reconnaissance and Enumeration

Before attempting any exploitation, I carried out reconnaissance and enumeration to profile the target's infrastructure and discover exposed entry points. This groundwork directly fed into several of the findings documented in Section 4.

3.1 Infrastructure Fingerprinting

WHOIS, nslookup, and DNSRecon queries were run against medirozahospital.com, resolving the domain to 199.188.201.16, hosted via NameCheap / cPanel-based shared hosting. HTTP header inspection via curl identified the web server as LiteSpeed, and an Nmap scan showed only ports 80 and 443 exposed, with an Imperva WAF observed in the network path.



```
kali@kali:~$ whois medirozahospital.com
Domain Name: MEDIROZAHOSPITAL.COM
Registry Domain ID: 312326963_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.namecheap.com
Registrar URL: http://www.namecheap.com
Updated Date: 2026-08-14T20:51:42Z
Creation Date: 2026-08-14T20:51:24Z
Registrar: NameCheap, Inc.
Registrar IANA ID: 1866
Registrar Abuse Contact Email: abuse@namecheap.com
Registrar Abuse Contact Phone: +1.663382107
Domain Status: clientTransferProhibited https://icann.org/epp/clientTransferProhibited
Name Server: DNS1.NAMECHEAPHHOSTING.COM
Name Server: DNS2.NAMECHEAPHHOSTING.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-08-09T03:10:19Z ooo

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois
database through the use of electronic processes that are high-volume and
automated except as reasonably necessary to register domain names or
modify existing registrations; the data in Verisign Global Registry
Services ("Verisign") whois database is provided by Verisign for
information purposes only, and to assist persons in obtaining information
about or related to a domain name registration record. Verisign does not
guarantee its accuracy. By submitting a Whois query, you agree to abide
by the following terms of use: You agree that you may use this data only
for lawful purposes and that under no circumstances will you use this data
to: (1) allow, enable, or otherwise support the transmission of mass
unsolicited, commercial advertising or solicitation via email, telephone,
or facsimile; or (2) enable high volume, automated, electronic processes
that apply to Verisign (or its computer systems). The compilation,
repackaging, dissemination or other use of this data is expressly
prohibited without the prior written consent of Verisign. You agree not to
use electronic processes that are automated and high-volume to access or
query the Whois database except as reasonably necessary to register
domain names or modify existing registrations. Verisign reserves the right
to restrict your access to the Whois database in its sole discretion to ensure
```

Figure 1: WHOIS reconnaissance against medirozahospital.com.

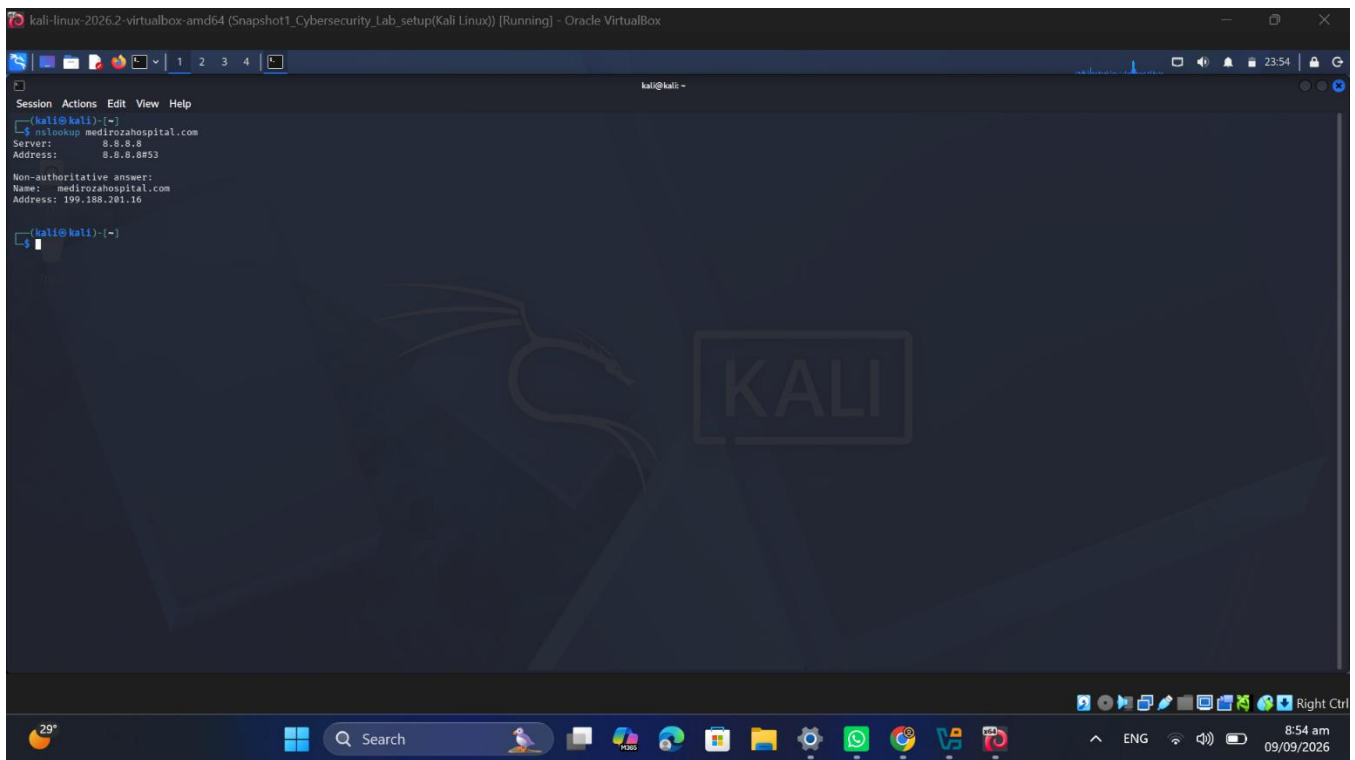


Figure 2: DNS resolution confirming the target IP address.

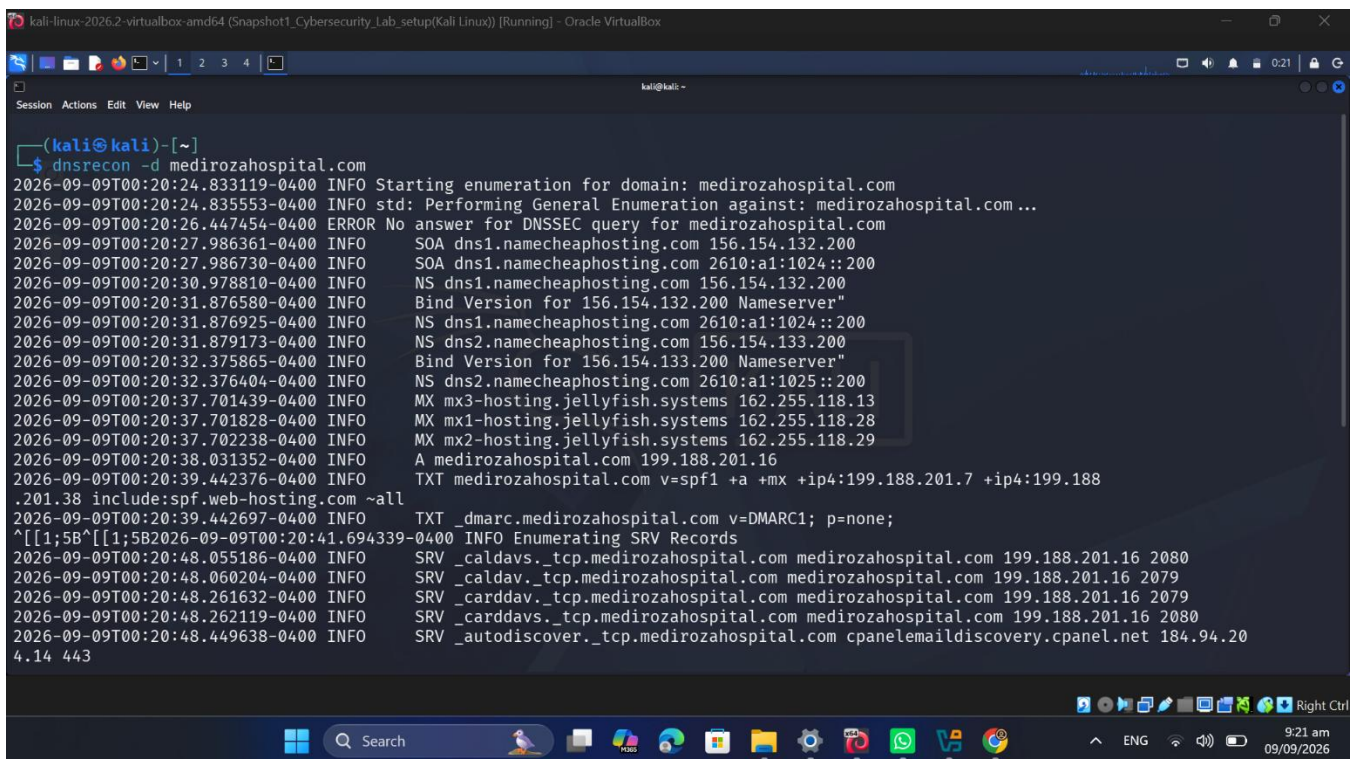


Figure 3: DNSRecon confirming cPanel-based hosting infrastructure.

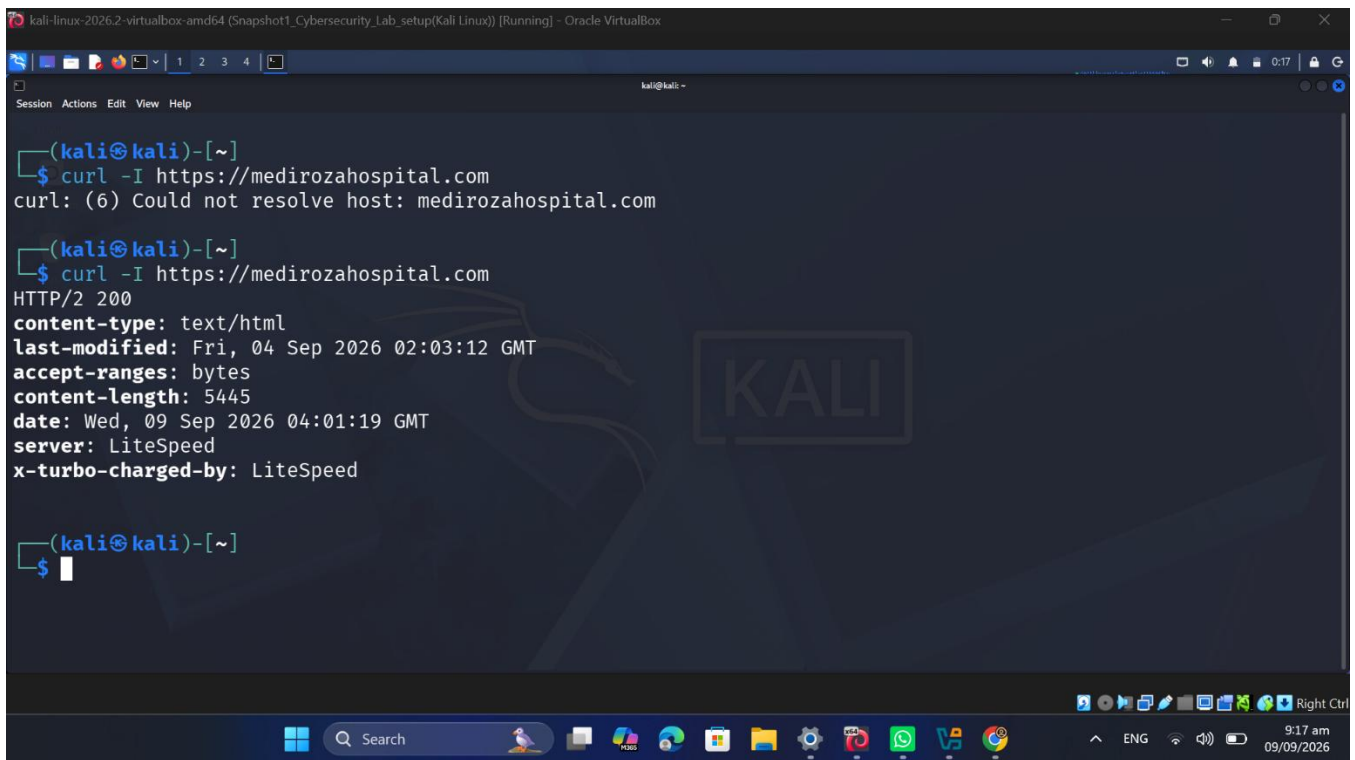


Figure 4: HTTP header inspection via curl identifying the LiteSpeed server.

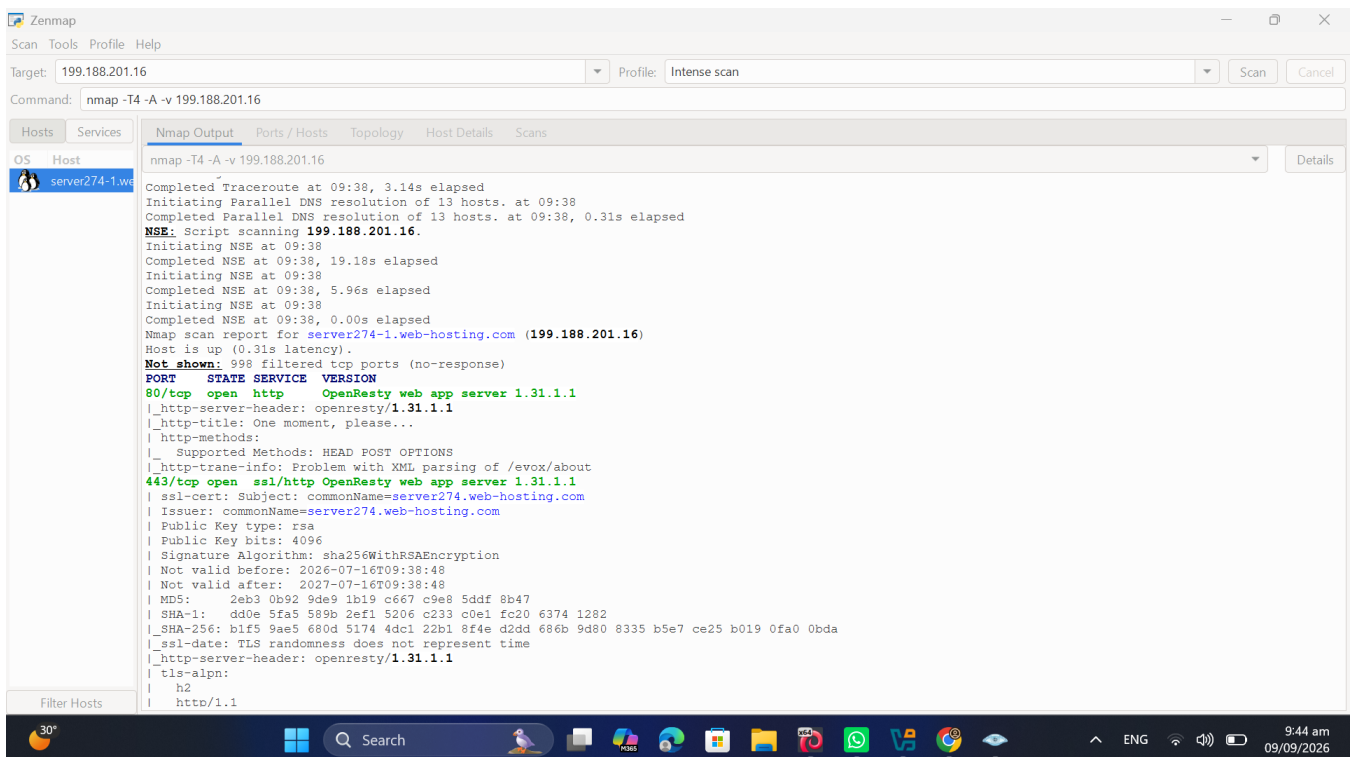
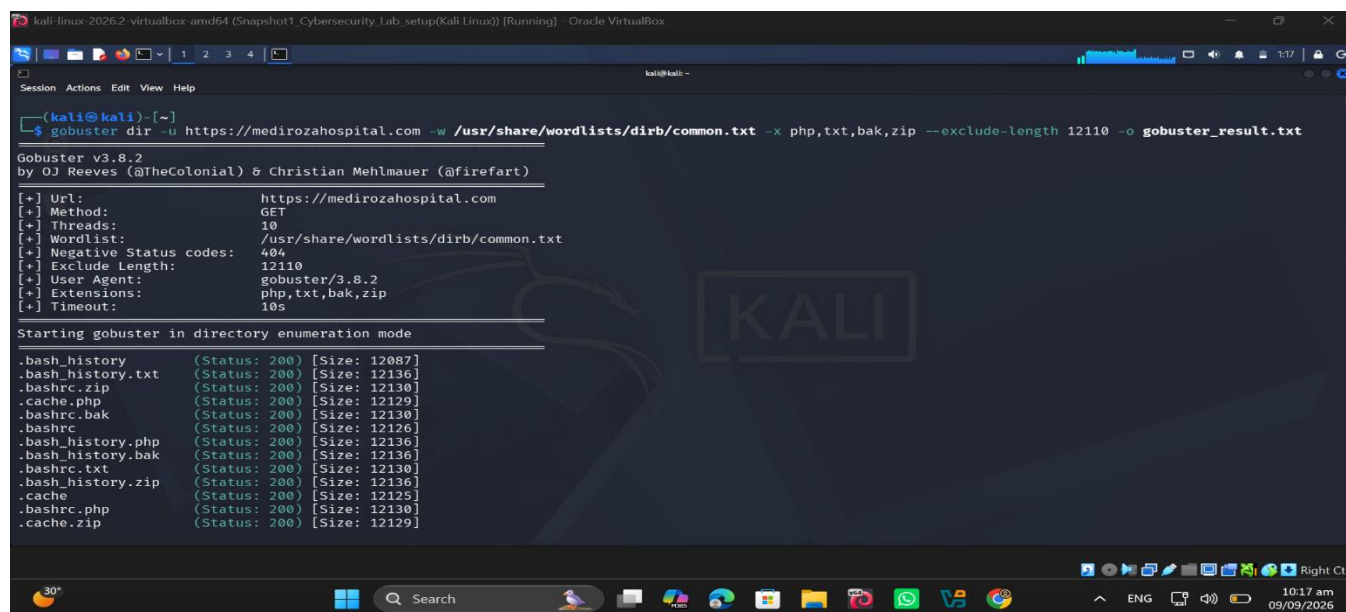


Figure 5: Zenmap scan showing only ports 80/443 exposed.

3.2 Directory and File Enumeration

Gobuster was used to enumerate hidden directories and files on the web root. An initial run returned unreliable results due to a soft-404 (wildcard) response being served with HTTP 200 for non-existent paths; this was corrected by excluding the wildcard response length.

```
gobuster dir -u https://medirozahospital.com -w
/usr/share/wordlists/dirb/common.txt -x php,txt,bak,zip --exclude-length
<wildcard_length> -o gobuster_result.txt
```



```
kali@kali:~$ gobuster dir -u https://medirozahospital.com -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak,zip --exclude-length 12110 -o gobuster_result.txt

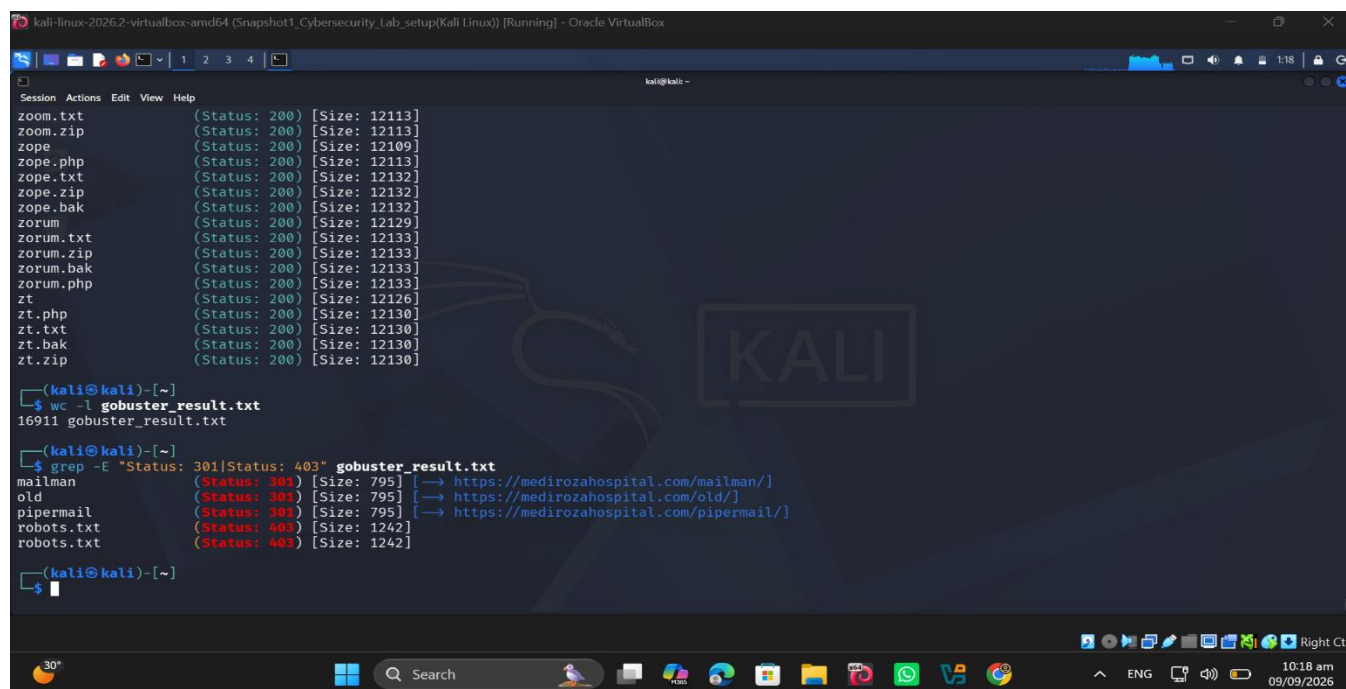
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             https://medirozahospital.com
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] Exclude Length:  12110
[+] User Agent:      gobuster/3.8.2
[+] Extensions:     php,txt,bak,zip
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

.bash_history      (Status: 200) [Size: 12087]
.bash_history.txt  (Status: 200) [Size: 12136]
.bashrc.zip        (Status: 200) [Size: 12130]
.cache.php         (Status: 200) [Size: 12129]
.bashrc.bak        (Status: 200) [Size: 12130]
.bashrc            (Status: 200) [Size: 12126]
.bash_history.php  (Status: 200) [Size: 12136]
.bash_history.bak  (Status: 200) [Size: 12136]
.bashrc.txt        (Status: 200) [Size: 12130]
.bash_history.zip  (Status: 200) [Size: 12136]
.cache             (Status: 200) [Size: 12125]
.bashrc.php        (Status: 200) [Size: 12130]
.cache.zip         (Status: 200) [Size: 12129]
```

Figure 6: Gobuster configured to exclude the wildcard/soft-404 response length.



```
kali@kali:~$ wc -l gobuster_result.txt
16911 gobuster_result.txt

kali@kali:~$ grep -E "Status: 403" gobuster_result.txt
mailman (Status: 403) [Size: 795] [→ https://medirozahospital.com/mailman/]
old      (Status: 403) [Size: 795] [→ https://medirozahospital.com/old/]
pipemail (Status: 403) [Size: 795] [→ https://medirozahospital.com/pipemail/]
robots.txt (Status: 403) [Size: 1242]
robots.txt (Status: 403) [Size: 1242]
```

Figure 7: Filtered Gobuster results revealing /old/, /mailman/, /pipemail/, and robots.txt.

3.3 robots.txt Disclosure

robots.txt was queried to identify indexing restrictions intended for web crawlers.

```
curl -s https://medirozahospital.com/robots.txt
```

RESPONSE

```
User-agent: * Disallow: /patient/ Disallow: /staff/ Disallow: /old/
```

The file explicitly disallowed the routes /patient/, /staff/, and /old/. Instead of restricting access, this revealed the internal directory structure directly to attackers — a classic information disclosure anti-pattern that guided the remainder of this assessment.

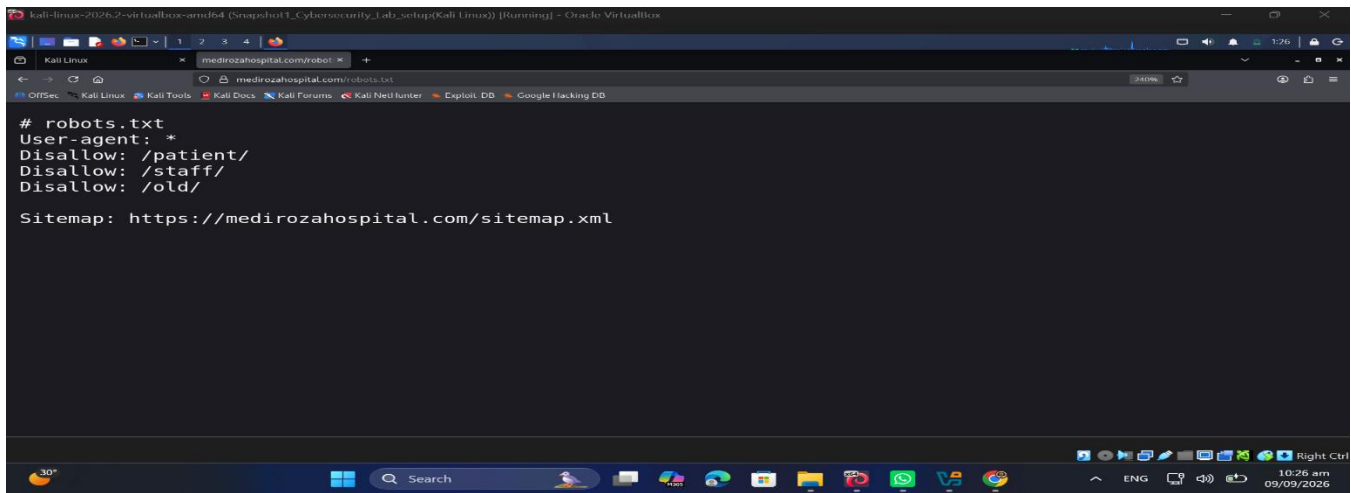


Figure 8: robots.txt disclosing sensitive, restricted directories.

The /staff/ path was found to host a staff authentication portal, with directory listing also enabled but only exposing the login.php file itself.

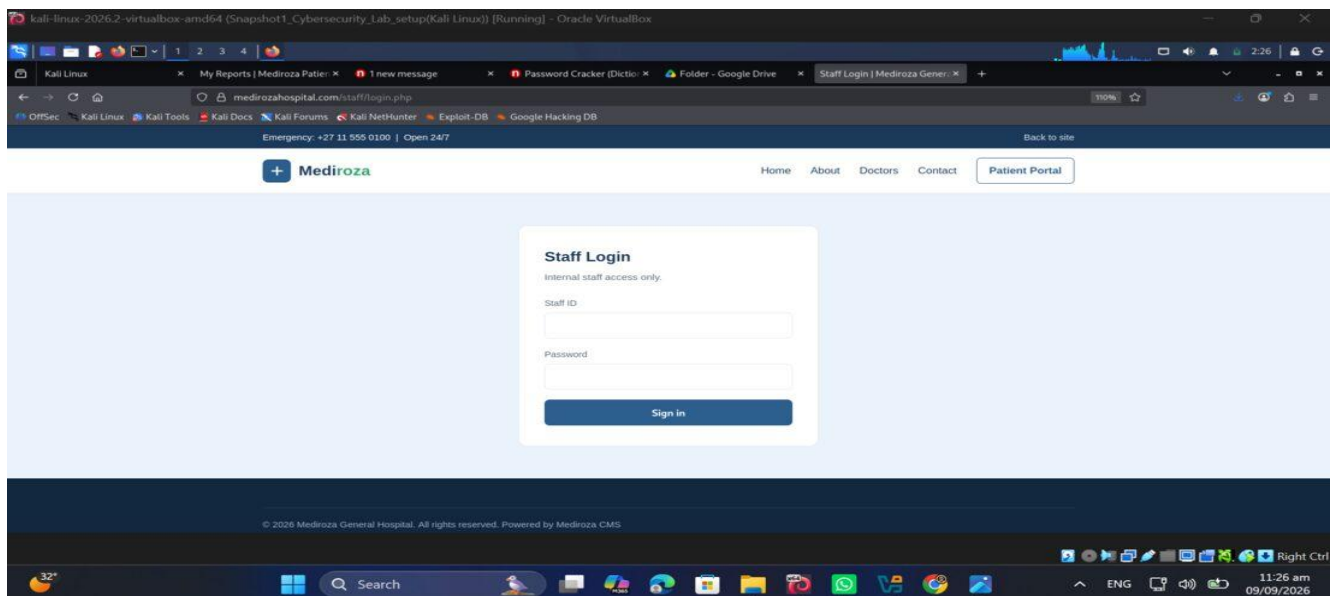


Figure 9: Staff Login portal at /staff/login.php.

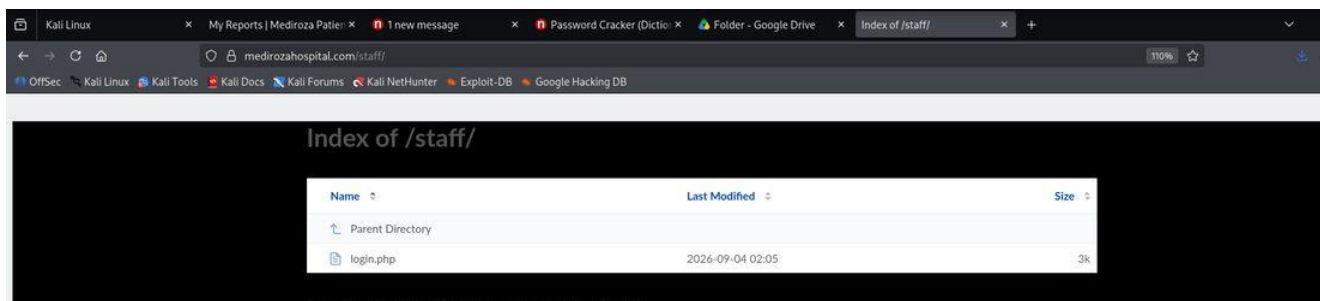


Figure 10: Directory listing at /staff/, exposing only login.php.

4. Findings and Proof of Exploitation

4.1 Summary Table

#	Vulnerability	Location	Risk
1	Username enumeration on login page	patient/login.php	Medium
2	SQL Injection login bypass	patient/login.php	Critical
3	Encrypted PDFs accessible after login bypass	patient/portal.php	High
4	Weak PDF passwords crackable with a wordlist	patient_report_*.pdf	High
5	Sensitive metadata left in patient PDF files	patient_report_3.pdf	Medium
6	Forgotten backup folder with directory listing enabled	old/	Critical
7	Confidential staff salaries and shareholder data in plain text	old/mediroza_db_backup_2019.sql	Critical

4.2 Finding 1 — Username Enumeration

Risk Rating: Medium

Location: patient/login.php

Description

Username enumeration occurs when a login page reveals whether a username exists or not by showing different error messages for a wrong username versus a wrong password. A secure login should always show the same generic message for both cases, to prevent an attacker from confirming valid usernames before attempting further attacks.

Steps Taken

I opened the patient portal login page and submitted a random, invalid username with an arbitrary password.

```
username: test    password: test
```

RESPONSE

Username not found

This username-specific response, rather than a generic authentication failure, confirms that the application discloses whether a submitted username exists, enabling an attacker to build a list of valid accounts before attempting password or injection attacks.

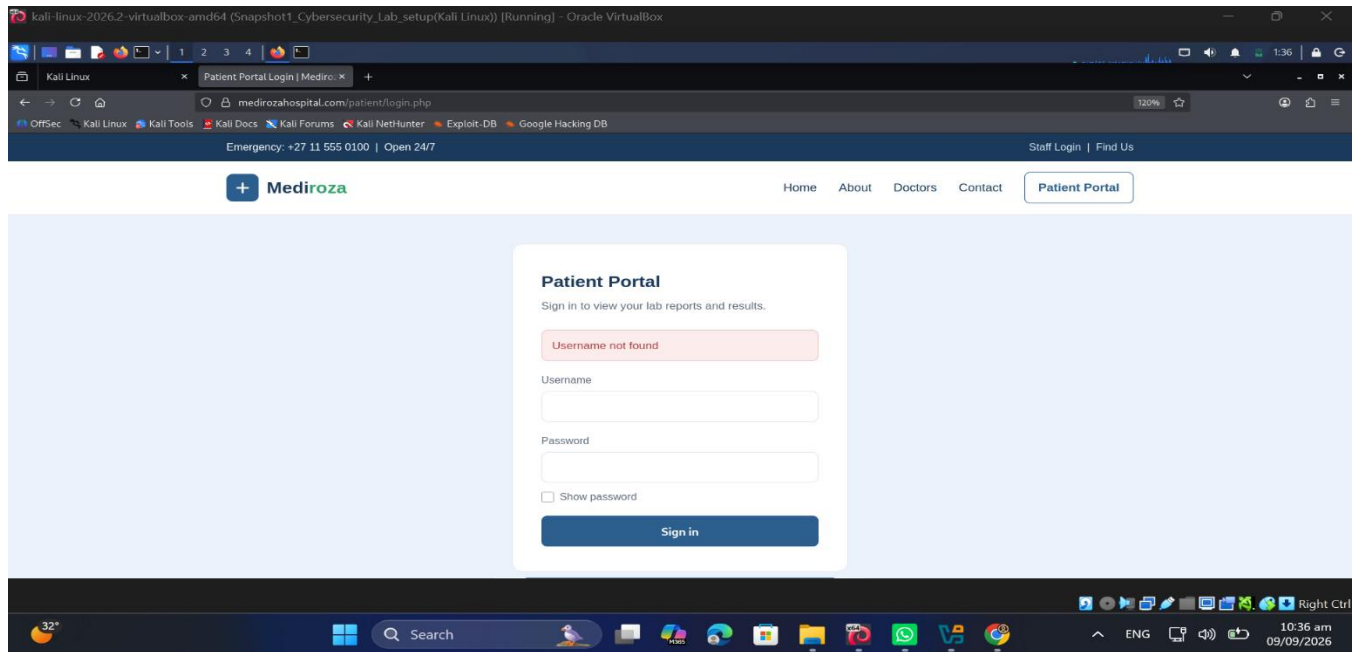


Figure 11: Patient portal returning a username-specific error, confirming a username enumeration weakness.

4.3 Finding 2 — SQL Injection Login Bypass

Risk Rating: Critical

Location: patient/login.php

Description

SQL Injection is a vulnerability where an application places user input directly inside a database query without sanitising it first. An attacker can insert SQL code into the input field to change the behaviour of the query. In this case, I was able to comment out the password check entirely and log in without knowing any valid password.

Steps Taken

I tested the username field for SQL Injection using a payload containing a logic-altering condition:

```
username: ' OR 1=1-- - password: '
```

RESPONSE

Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' OR 1=1-- -' at line 1

The verbose database error confirmed the username field was directly concatenated into the backend SQL query without sanitisation. After refining the payload, the following authentication-bypass string was submitted with the password field left blank:

```
username: admin'-- - password: (blank)
```

The double-dash comment sequence truncated the query before the password check could be evaluated. The application granted a fully authenticated session and redirected to the restricted patient portal — without any valid credentials.

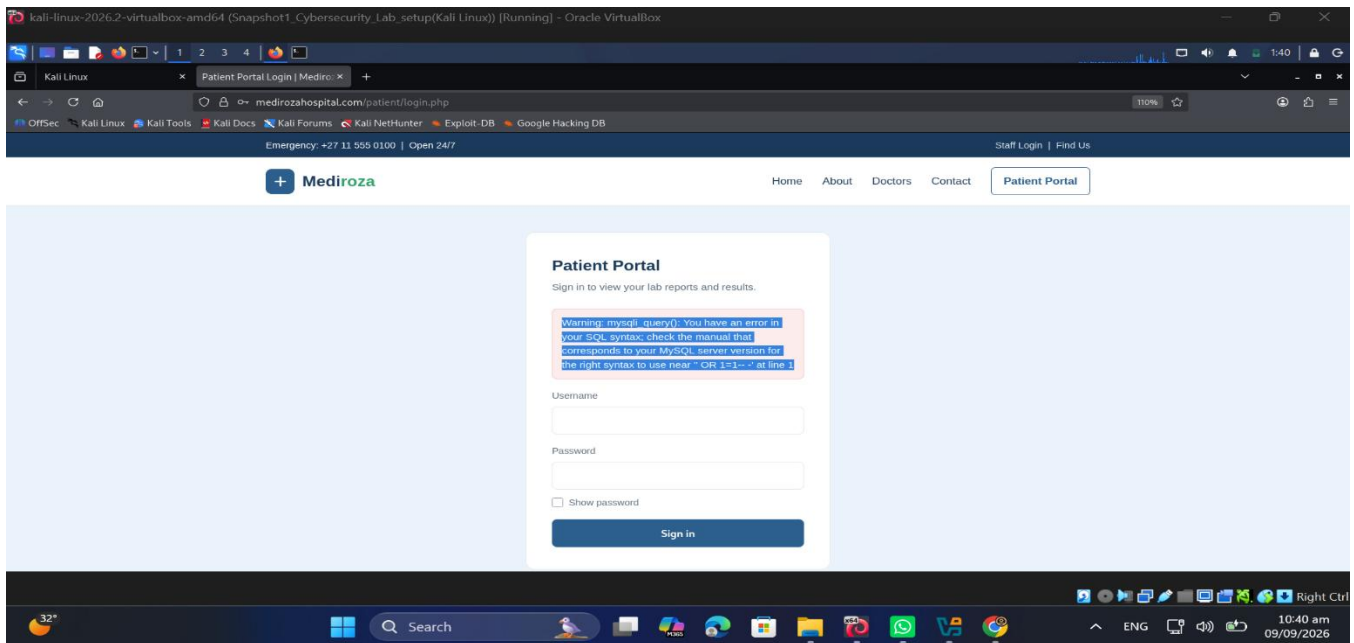


Figure 12: Verbose MySQL syntax error confirming unsanitised query concatenation.

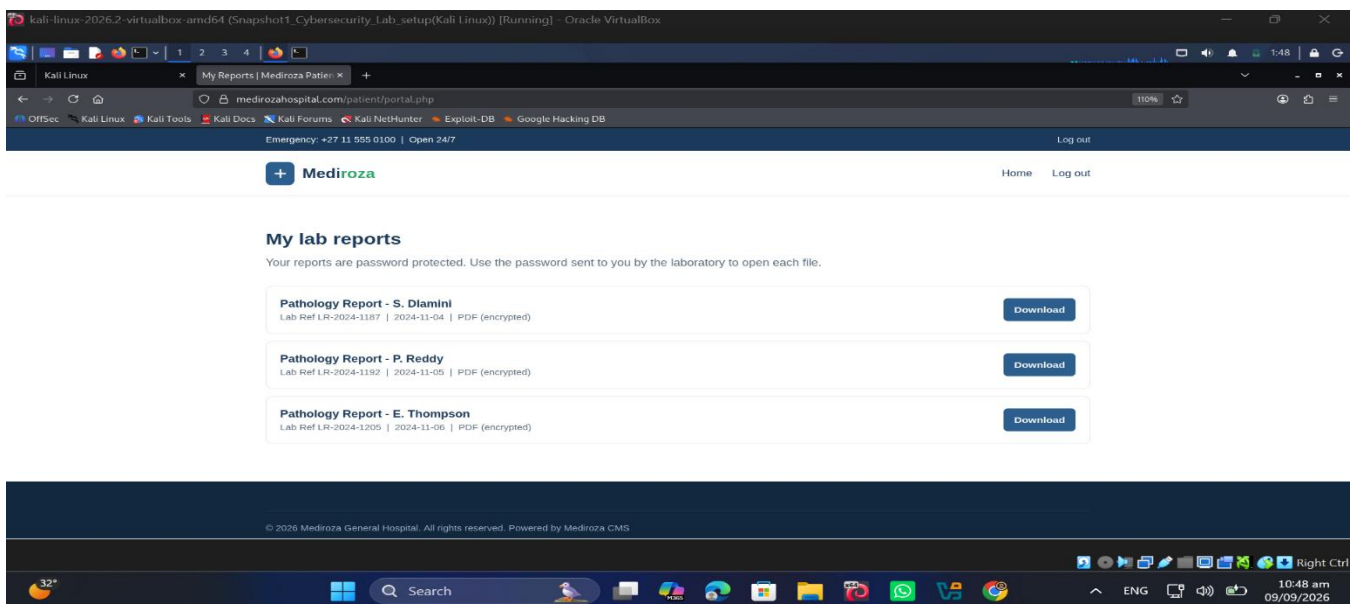


Figure 13: Successful authentication bypass, redirected to the patient portal dashboard.

4.4 Finding 3 — Confidential PDFs Accessible After Login Bypass

Risk Rating: High

Location: patient/portal.php

Description

After gaining unauthorised access to the portal through SQL Injection, I found three patient lab report PDFs available for download. These are confidential medical documents that should only be accessible to the named patients and their treating physicians.

Steps Taken

The compromised portal session listed three downloadable reports, which I downloaded to the local testing machine:

- Pathology Report — S. Dlamini (Lab Ref LR-2024-1187)
- Pathology Report — P. Reddy (Lab Ref LR-2024-1192)
- Pathology Report — E. Thompson (Lab Ref LR-2024-1205)

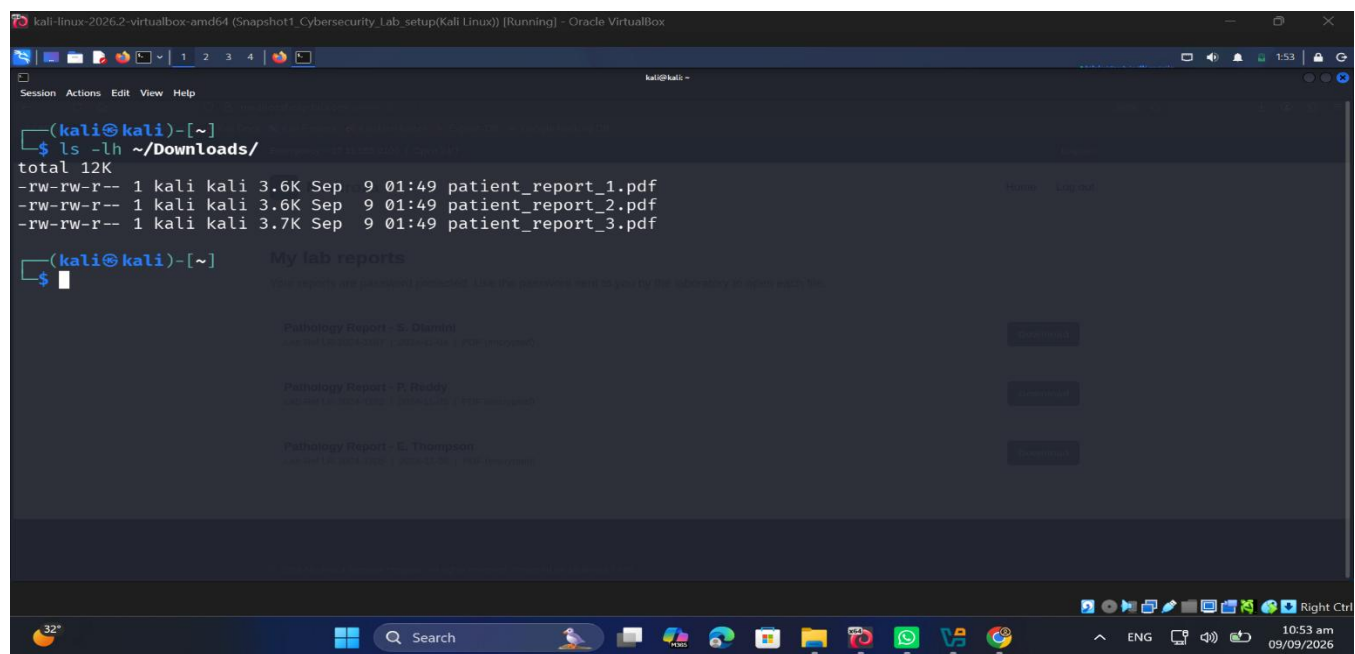


Figure 14: The three retrieved patient pathology PDF reports, confirmed downloaded to disk.

4.5 Finding 4 — Weak PDF Passwords Crackable with a Wordlist

Risk Rating: High

Location: patient_report_1.pdf, patient_report_2.pdf, patient_report_3.pdf

Description

All three PDFs were password protected using Standard V2.3 (128-bit) PDF encryption. However, the passwords were weak and appeared in commonly available password wordlists, making them trivial to crack using automated tools.

Steps Taken

I used the Networkwalks Hash Calculator to extract a crackable (\$pdf\$ / pdf2john-compatible) hash from each PDF, then ran each hash through the Networkwalks Password Cracker.

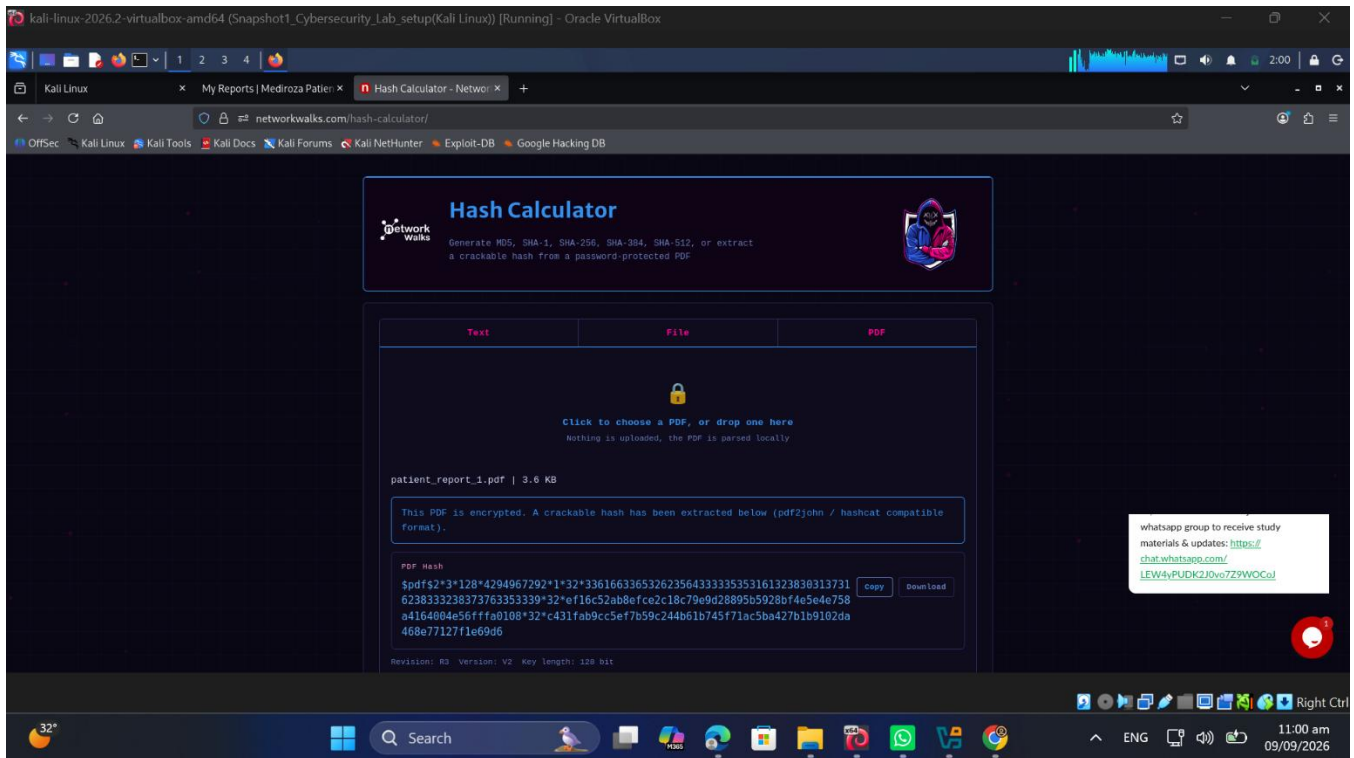


Figure 15: Extraction of a crackable \$pdf\$ hash from patient_report_1.pdf.

Reports 1 and 2 cracked immediately using the built-in 100-word default wordlist.

RESPONSE

patient_report_1.pdf → 123456 patient_report_2.pdf → password

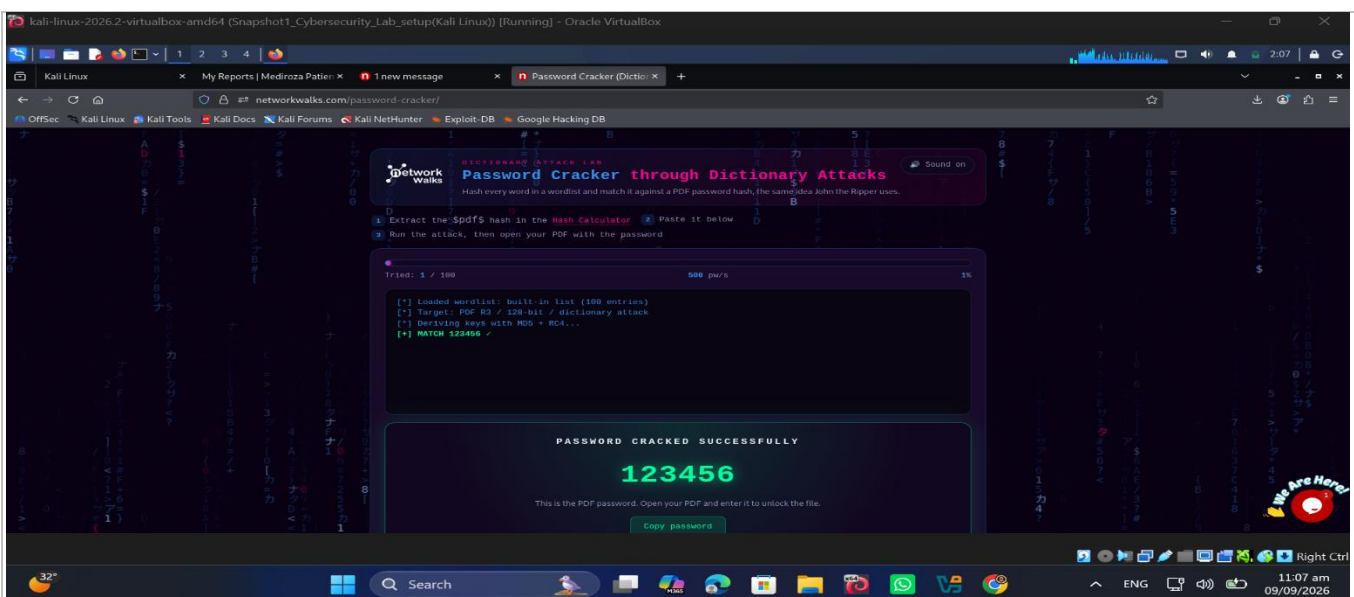


Figure 16: Dictionary attack recovering the password '123456' for patient_report_1.pdf.

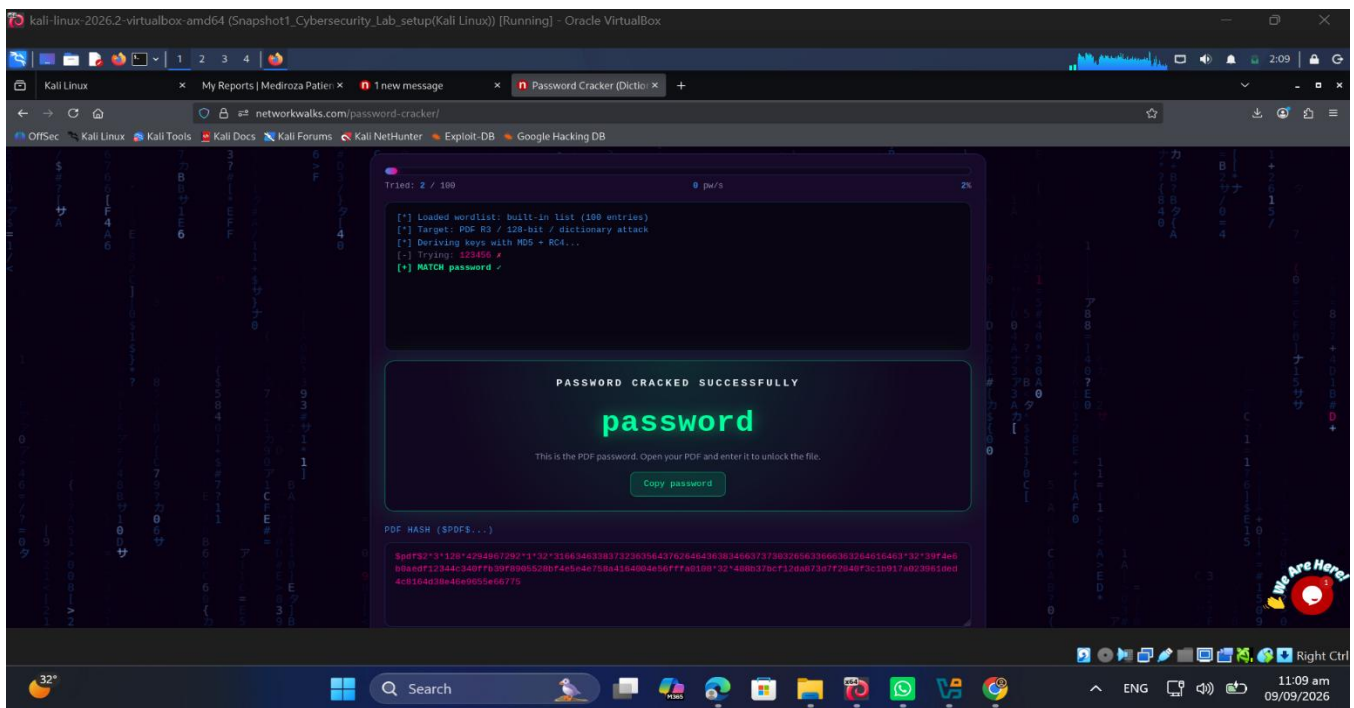


Figure 17: Dictionary attack recovering the password 'password' for patient_report_2.pdf.

Report 3 did not crack with the built-in list. I switched to a larger wordlist (the JTR default password list) and re-ran the attack.

RESPONSE

patient_report_3.pdf → !@#%\$^&

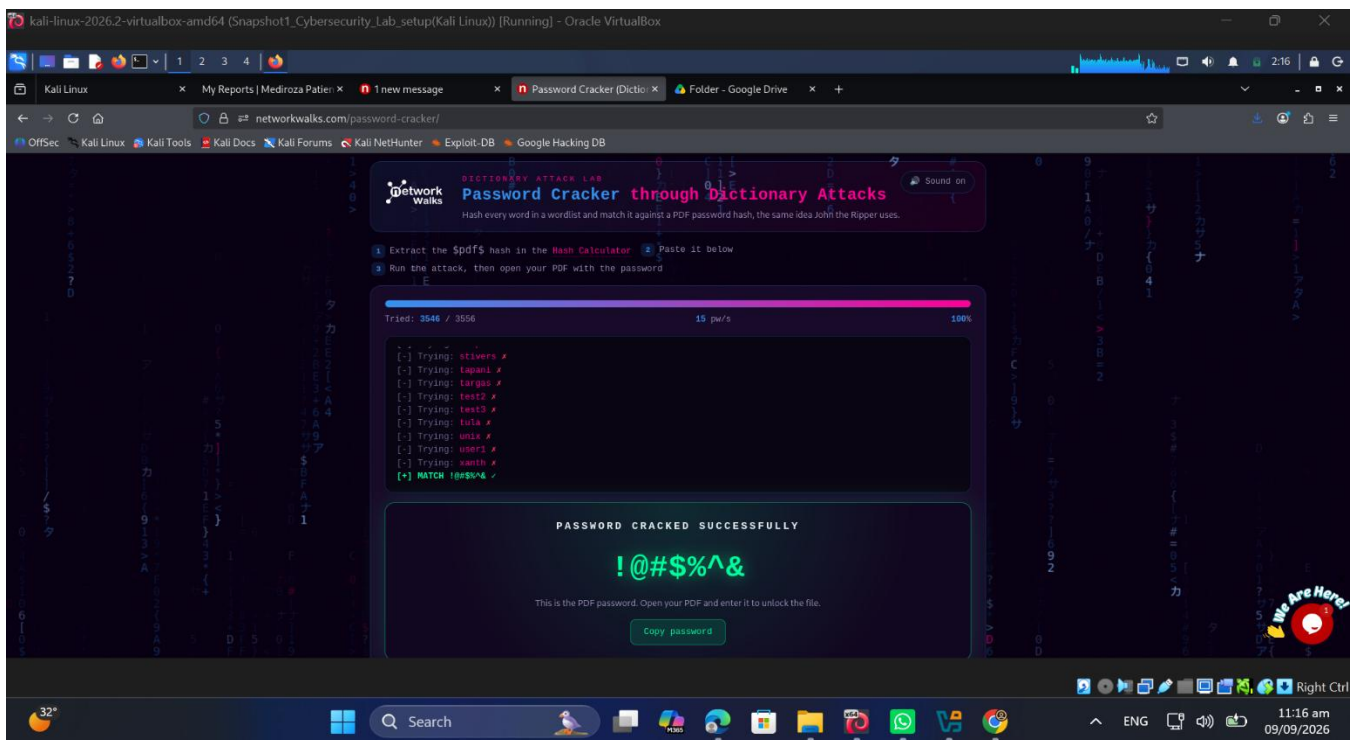


Figure 18: Dictionary attack recovering the password '!@#%\$^&' for patient_report_3.pdf using the larger wordlist.

I then opened each PDF with its cracked password and read the confidential patient medical data inside, including full names, dates of birth, patient IDs, referring physicians, and complete laboratory test results with clinical HIGH/Normal flags.



Figure 19:Decrypted pathology report for patient Sipho Dlamini.

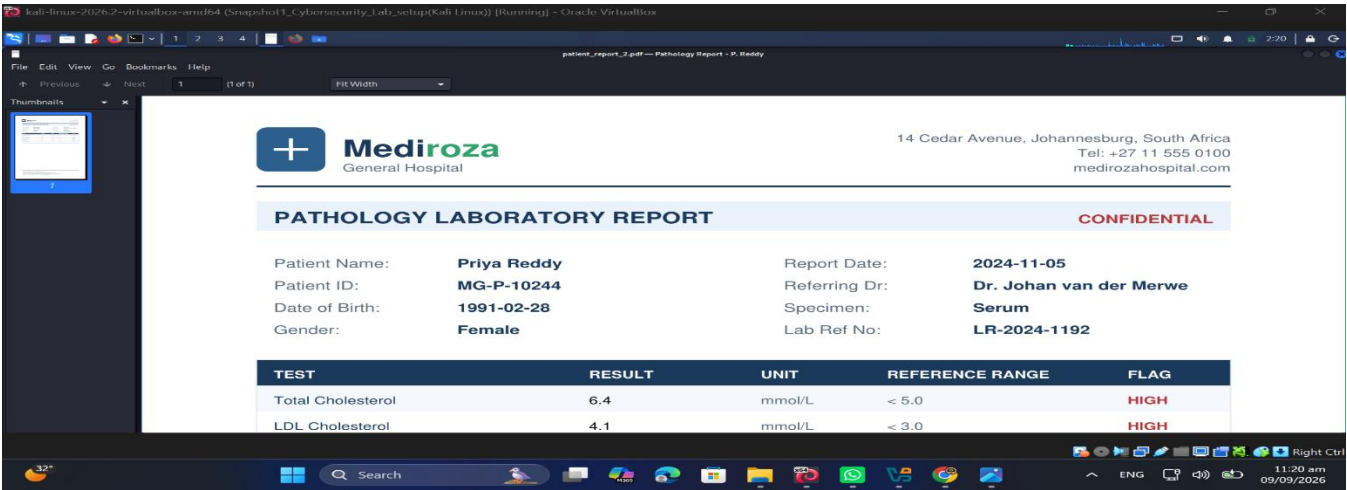


Figure 20:Decrypted pathology report for patient Priya Reddy, showing cholesterol results flagged HIGH.

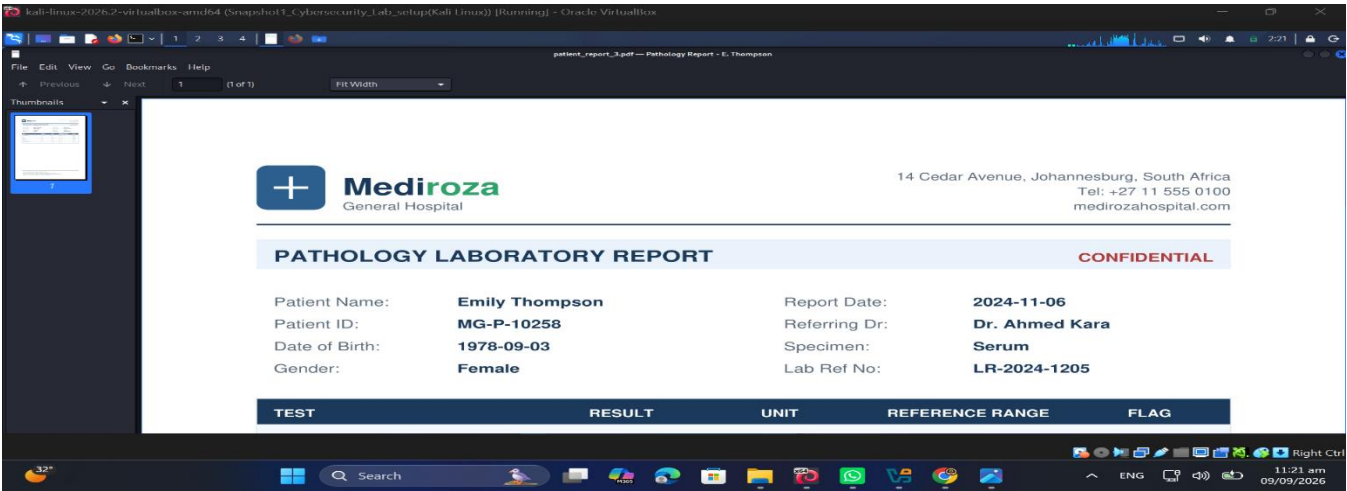


Figure 21:Decrypted pathology report for patient Emily Thompson.

4.6 Finding 5 — Sensitive Metadata in Patient PDF Files

Risk Rating: Medium

Location: patient_report_3.pdf

Description

PDF files store hidden metadata such as the author name, creation tool, and custom comments. This metadata is not visible when simply reading the document but can be extracted using tools like ExifTool. In this case, report 3 contained an internal staff comment left by an IT administrator that pointed directly to a confidential backup location on the server.

Steps Taken

Using the recovered password, I ran ExifTool directly against the encrypted PDF:

```
exiftool -password '!@#$$%^&' patient_report_3.pdf
```

RESPONSE

Author: j.malik Comments: DB backup moved to /old before site migration, do not delete

This comment was left by IT Systems Administrator Jameel Malik (j.malik) — a name later confirmed in the exposed staff database (Finding 7) — and independently revealed the exact location of a database backup on the server, corroborating the /old/ directory already flagged via robots.txt (Section 3.3).

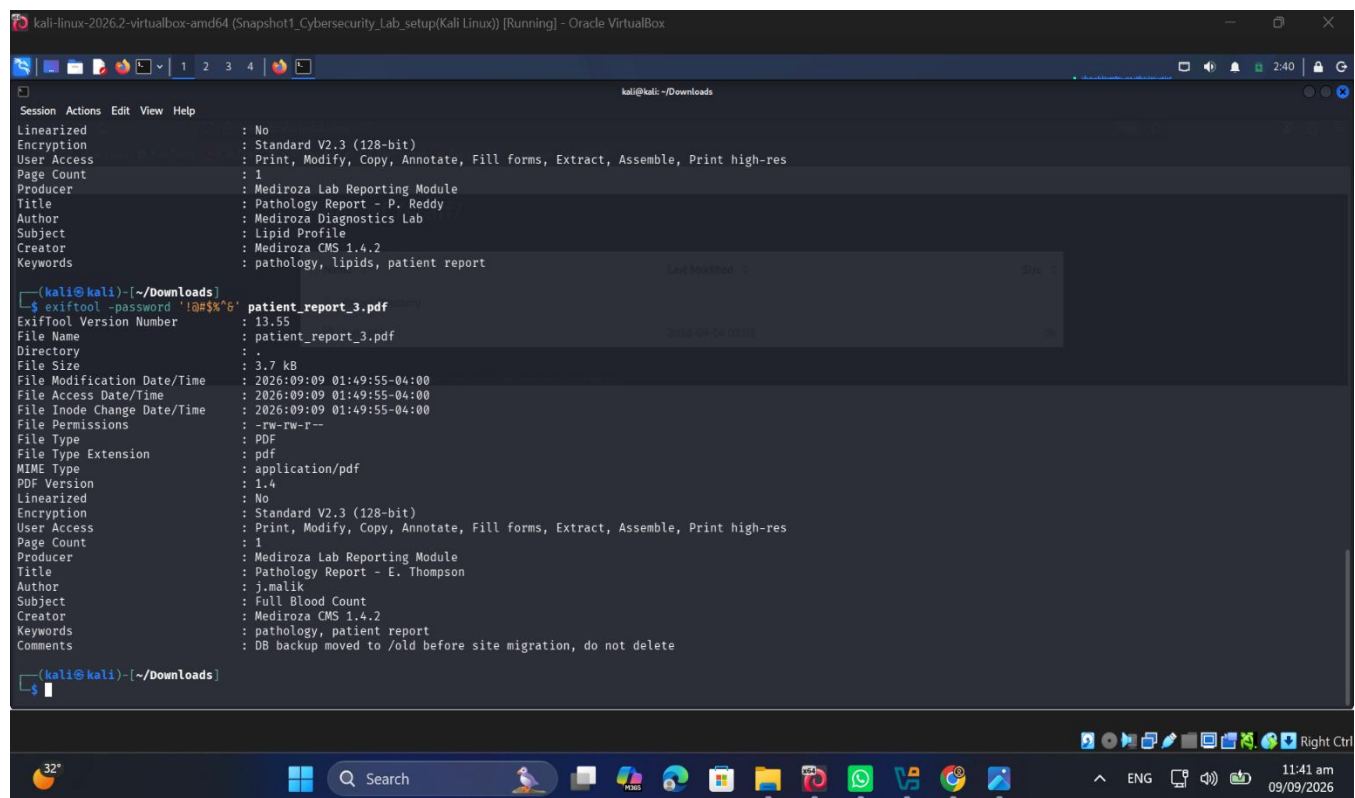


Figure 22: ExifTool output showing the Author field (j.malik) and the Comments field disclosing the /old/ backup location.

4.7 Finding 6 — Forgotten Backup Folder with Directory Listing Enabled

Risk Rating: **Critical**

Location: old/

Description

Directory listing is a web server misconfiguration that displays the contents of a folder like a file browser when no index page exists. The /old/ folder had directory listing enabled, exposing a database backup file to anyone who visited that URL. This folder had already been identified during reconnaissance from robots.txt (Section 3.3), and the metadata clue from Finding 5 independently confirmed it as the location of the backup.

Steps Taken

I opened the folder directly in the browser:

```
https://medirozahospital.com/old/
```

The directory listing was enabled, and the backup file was immediately visible: mediroza_db_backup_2019.sql. I downloaded it using wget:

```
wget https://medirozahospital.com/old/mediroza_db_backup_2019.sql
```

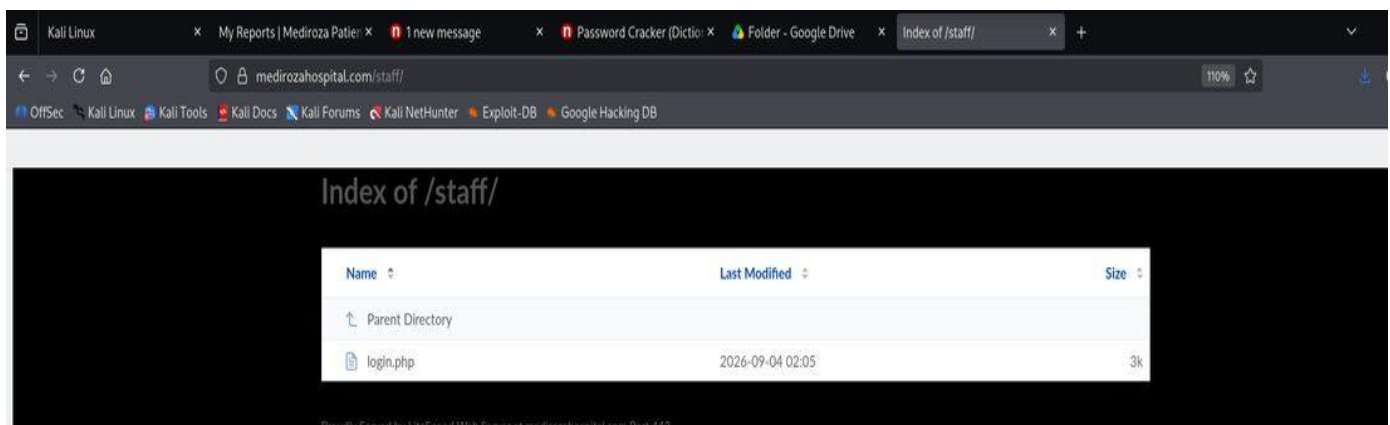


Figure 23: Directory listing enabled at /old/, exposing mediroza_db_backup_2019.sql without authentication.

4.8 Finding 7 — Confidential Staff Salaries and Shareholder Data in Plain Text

Risk Rating: **Critical**

Location: old/mediroza_db_backup_2019.sql

Description

The database backup contained highly sensitive information in plain text, including the full name, job title, department, work email address, phone number, national ID number, and monthly salary (in ZAR) for all 30 hospital employees. It also contained the names, share percentages, and share classes of all 10 shareholders of the hospital. This data was completely unprotected and accessible to anyone who visited the /old/ folder.

Steps Taken

I inspected the downloaded SQL file directly:

```
cat mediroza_db_backup_2019.sql
```

The file's own header explicitly labelled it as a confidential internal backup of the mediroza_hr database, containing a staff table (30 records) and a shareholders table (10 records).

Employee	Job Title	Monthly Salary (ZAR)
Dr. Johan van der Merwe	Medical Director	R 160,000
Sarah Botha	Chief Financial Officer	R 152,000
Dr. Rajesh Naidoo	Chief Pathologist	R 138,000
(27 further staff records)	Various departments	R 19,000 – R 135,000

Shareholder	Share %	Class
Dr. Rajesh Naidoo	18.0%	Ordinary
Cedar Health Holdings (Pty) Ltd	15.0%	Ordinary
Dr. Johan van der Merwe	12.0%	Ordinary
(7 further shareholder records)	4.0% – 11.0%	Ordinary / Preferential

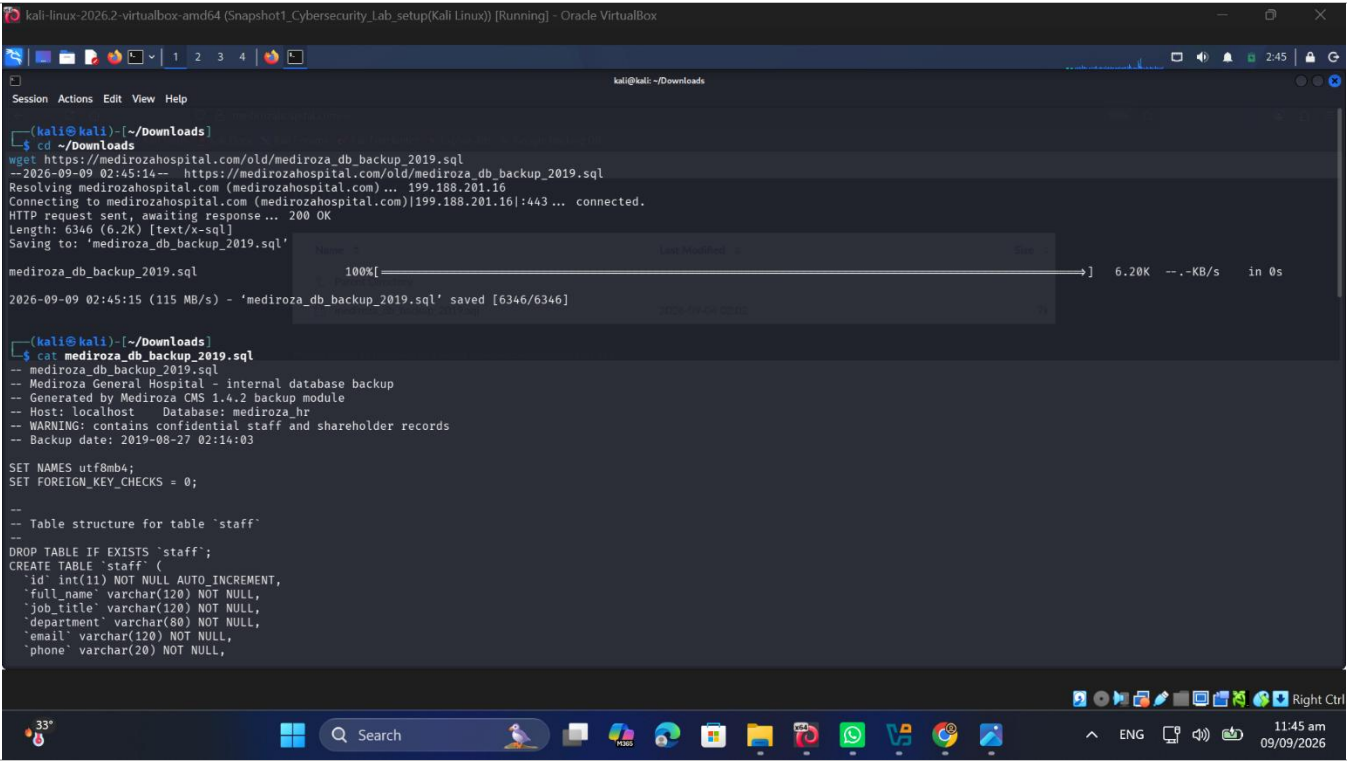
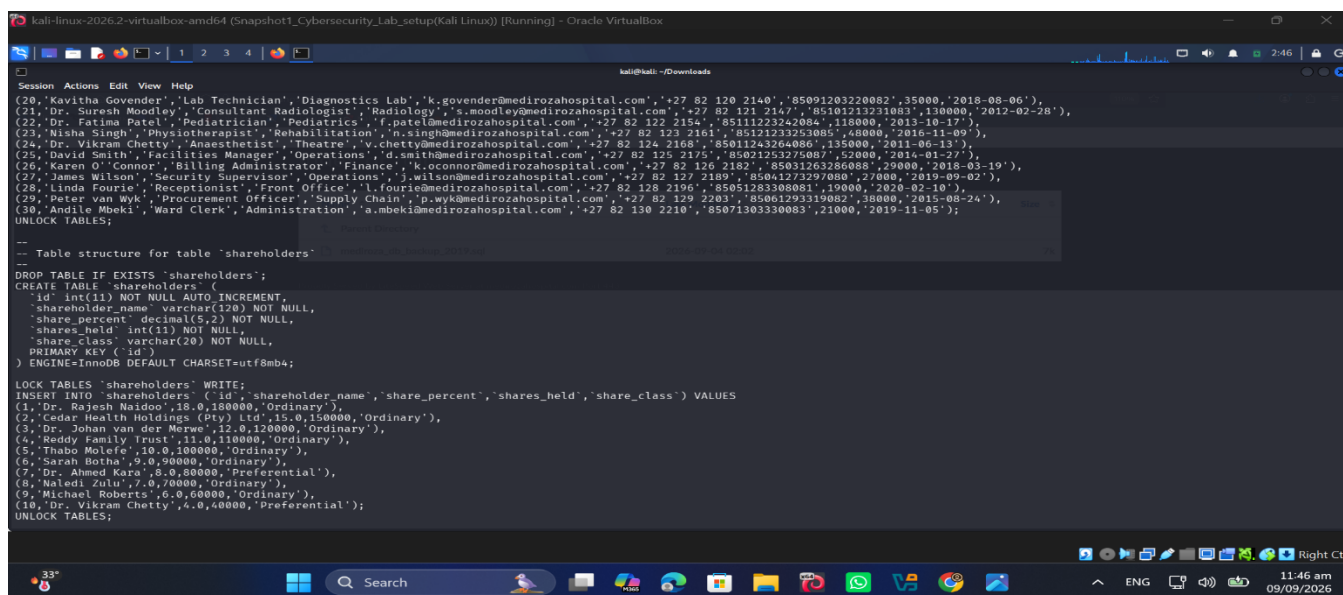


Figure 24: Direct inspection of mediroza_db_backup_2019.sql, revealing the staff table structure and header warning.



```
kali@kali:~/Downloads
--
-- Table structure for table `shareholders`
--
DROP TABLE IF EXISTS `shareholders`;
CREATE TABLE `shareholders` (
  `id` int(11) NOT NULL AUTO INCREMENT,
  `shareholder_name` varchar(120) NOT NULL,
  `share_percent` decimal(5,2) NOT NULL,
  `shares_held` int(11) NOT NULL,
  `share_class` varchar(20) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

LOCK TABLES `shareholders` WRITE;
INSERT INTO `shareholders` (`id`,`shareholder_name`,`share_percent`,`shares_held`,`share_class`) VALUES
(1,'Dr. Rajesh Naidoo',10.0,100000,'Ordinary'),
(2,'Cedar Health Holdings (Pty) Ltd',15.0,150000,'Ordinary'),
(3,'Dr. Johan van der Merwe',12.0,120000,'Ordinary'),
(4,'Reddy Family Trust',11.0,110000,'Ordinary'),
(5,'Thabo Molefe',10.0,100000,'Ordinary'),
(6,'Sarah Botha',9.0,90000,'Ordinary'),
(7,'Dr. Ahmed Kara',8.0,80000,'Preferential'),
(8,'Naledi Zulu',7.0,70000,'Ordinary'),
(9,'Michael Roberts',6.0,60000,'Ordinary'),
(10,'Dr. Vikram Chetty',5.0,50000,'Preferential');
UNLOCK TABLES;
```

Figure 25: Full shareholders table recovered from the exposed database backup.

I also confirmed that the Author field in the PDF metadata (Finding 5), j.malik, corresponds to Jameel Malik, the IT Systems Administrator listed in this staff table — connecting Findings 5, 6, and 7 into a single, coherent attack chain.

A due-diligence check was also performed against sibling paths (e.g. /patient/reports/), which correctly returned HTTP 403 Forbidden — confirming the exposure was isolated to the legacy /old/ directory, and that access controls were applied inconsistently across the environment.

5. Full Attack Chain Summary

The following is the complete path from zero access to full data exposure, showing how each finding led to the next.

- Step 1: robots.txt revealed three hidden folders: /patient/, /staff/, and /old/.
- Step 2: The patient login page returned a username-specific 'Username not found' error for invalid accounts, confirming username enumeration was possible. (Finding 1)
- Step 3: A single-quote-based payload in the username field triggered a verbose database error, confirming SQL Injection. (Finding 2)
- Step 4: The payload admin'-- bypassed the login entirely and gave access to the patient portal. (Finding 2)
- Step 5: Three confidential patient PDF lab reports were downloaded from the portal. (Finding 3)
- Step 6: The Networkwalks Password Cracker cracked all three PDF passwords using dictionary wordlists — reports 1 and 2 used the built-in list, report 3 required a larger wordlist. (Finding 4)
- Step 7: ExifTool on the decrypted report 3 revealed a hidden comment by j.malik pointing to /old/. (Finding 5)
- Step 8: The /old/ folder had directory listing enabled, exposing the database backup. (Finding 6)
- Step 9: The backup contained salaries of 30 employees and shareholding details of 10 shareholders in plain text. (Finding 7)

6. Risk Rating

Each finding was rated using a standard four-tier severity scale (Critical, High, Medium, Low), based on the sensitivity of the data exposed, the ease of exploitation, and the potential business impact to Mediroza General Hospital.

Finding	Milestone	Severity	CWE Reference
SQL Injection – Authentication Bypass on Patient Portal	M1	Critical	CWE-89
Username Enumeration on Patient Portal Login	M1	Low	CWE-203
Verbose Database Error Messages Disclosed to Users	M1	Medium	CWE-209
Weak / Guessable PDF Document Passwords	M2	High	CWE-521
Sensitive Path Disclosure via robots.txt	M3	Low	CWE-200
Directory Listing Enabled with Public Backup File (/old/)	M3	Critical	CWE-538
Unauthenticated Exposure of Staff PII, Salaries & Shareholder Equity	M3	Critical	CWE-200 / CWE-538
Sensitive Operational Comment Embedded in Patient-Facing PDF Metadata	M3	Medium	CWE-200

6.1 Justification Summary

- Critical: SQL Injection authentication bypass required no credentials, no special tooling, and directly exposed regulated patient data — full compromise achievable by any anonymous visitor.
- Critical: The exposed database backup contained unencrypted national ID numbers, salaries, and equity records for the entire organisation, retrievable via a single unauthenticated HTTP request.
- High: While exploitation required an extra step (offline cracking), all three PDF passwords fell to a wordlist attack within seconds, providing no meaningful protection in practice.
- Medium: Verbose error messages, username enumeration, and embedded metadata comments do not grant direct access on their own, but materially aid an attacker in identifying and exploiting the more severe findings above.

7. Recommendations and Remediation

7.1 Fix Username Enumeration

Change the login page to show a single, generic message for any failed login attempt, regardless of whether the username or password was wrong — for example: 'Invalid credentials. Please try again.'

7.2 Fix SQL Injection

Replace the current login query with a parameterised query or prepared statement. This separates the SQL code from the user input so that no amount of crafted input can change the query structure. This is the most important fix in this report.

```
// Safe example using a PHP PDO prepared statement $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?"); $stmt->execute([$username, $password]);
```

7.3 Fix PDF Access and Password Strength

#	Recommended Action
1	Move the PDF files outside the web root so they cannot be served directly by the web server.
2	Enforce access through a server-side script that checks authentication before serving any file.
3	If passwords are used, enforce a minimum length of 12 characters with a mix of uppercase, lowercase, numbers, and symbols.
4	Prefer strong, randomly generated per-document keys delivered via an authenticated channel over static, memorable passwords.

7.4 Strip PDF Metadata

Remove all metadata from files before distributing them. Staff notes or internal comments must never be stored inside files that leave the organisation.

```
exiftool -all= patient_report_3.pdf
```

7.5 Fix Directory Listing and Remove the Backup

#	Recommended Action
1	Disable directory listing on all folders (e.g. Options -Indexes in the Apache configuration or .htaccess file).
2	Delete the database backup from the /old/ folder immediately, and audit all web-accessible directories for other legacy or backup files.
3	Database backups must never be stored inside the web root — store them in a private, access-controlled, encrypted location outside the publicly accessible part of the server.
4	Treat the exposed staff and shareholder data as a personal-data incident and initiate the hospital's incident-response and notification procedures.

7.6 General Hardening

#	Recommended Action
1	Remove all sensitive path names from robots.txt — it is a public file and must never be relied upon for access control.
2	Conduct regular, scheduled penetration testing and vulnerability scanning of all public-facing infrastructure.
3	Implement centralised logging and alerting for sensitive file access (.sql, .bak, .zip) and repeated failed login attempts.
4	Provide secure-development training focused on OWASP Top 10 categories A03 (Injection) and A05 (Security Misconfiguration).

8. Conclusion

This assessment found a complete attack chain from the login page to highly sensitive internal data. No sophisticated tools or specialised knowledge were required — every vulnerability found in this report is well-known and has a well-established fix. I recommend the client address all Critical and High findings in Section 7 immediately before the system is used to store or serve real patient data.

Submitted by: Muhammad Ahsan

Cybersecurity Mentor: Waqas Karim, CCIE

Organisation: NetworkWalks

Week 4 Capstone Project

This report is produced as part of a controlled educational exercise by Networkwalks. The target has been authorised for security testing. These techniques must never be applied to any system without explicit written permission from the owner.

This report is submitted as part of the Networkwalks Cybersecurity Internship Week 4 Capstone Project. All testing was conducted in a controlled environment with written authorisation from the client. These techniques must never be applied to any system without explicit written permission from the owner.